GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY 3
CLIENT 3
INSTRUCTIONS 3
DEVELOPER 4
1. ALGORITHM CIPHER 4
2. CERTIFICATE GENERATION 4
3. DEPLOY CIPHER 4
4. SECURE COMMUNICATIONS 6
5. SECONDARY TESTING 6
6. FUNCTIONAL TESTING 8
7. SUMMARY 8
8. INDUSTRY STANDARD BEST PRACTICES 8

Document Revision History

Version	Date	Author	Comments
1.0	02-22-2026	Anneshia Mcknight	

Client

Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

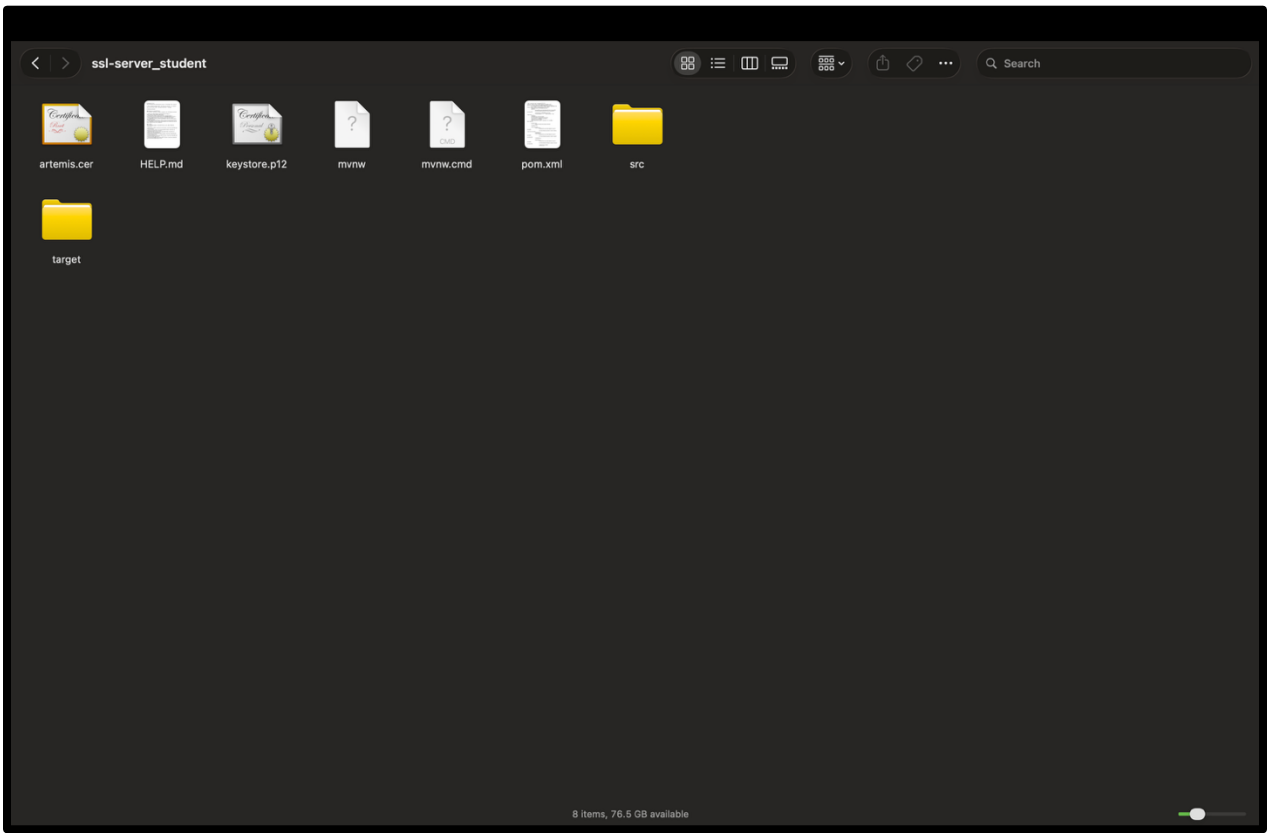
Developer
Anneshia Mcknight

1. Algorithm Cipher

SHA-256 was used as the most modern algorithm, producing a fixed 32-byte strong sequence. Another advantage is that SHA-256 does not use keys, and the result is determined only by the input data — the algorithm is based on bitwise operations, ensuring good cryptographic strength.

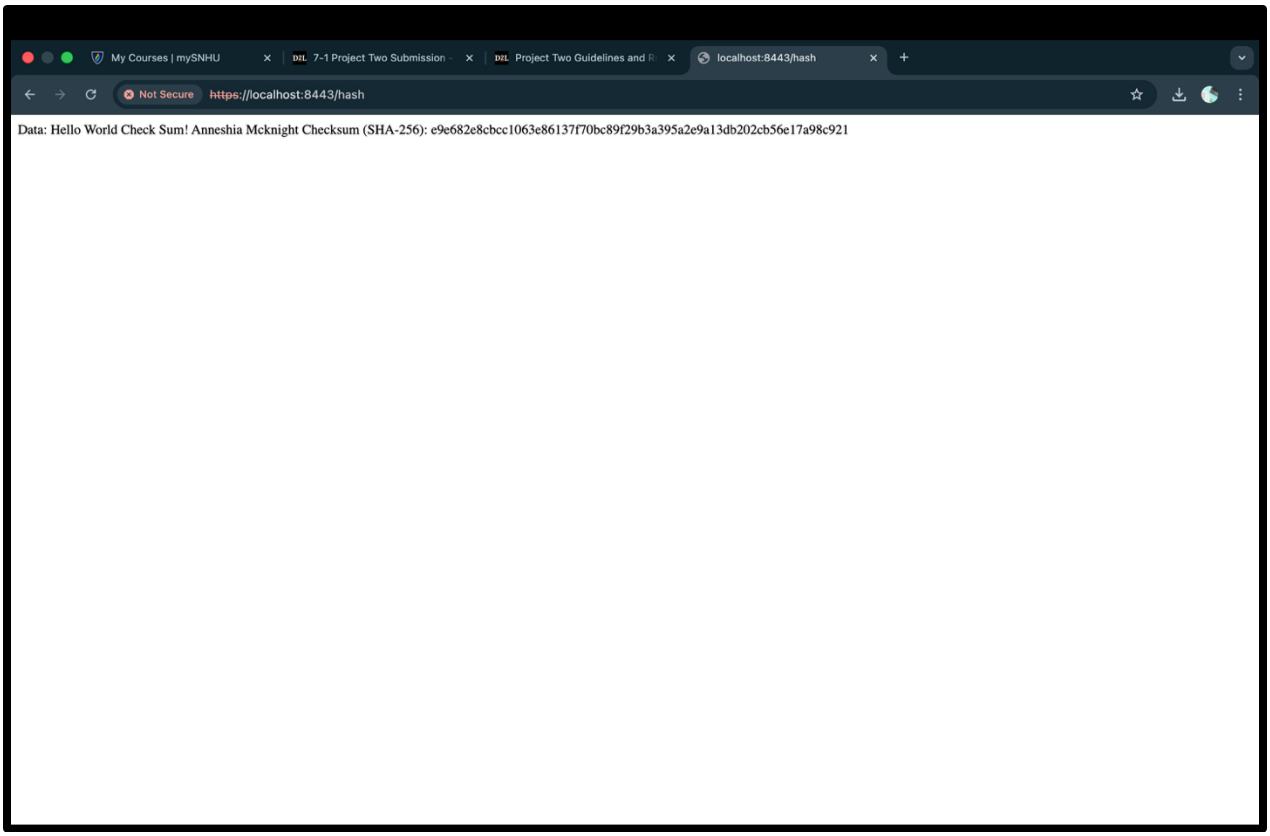
2. Certificate Generation

Insert a screenshot below of the CER file.



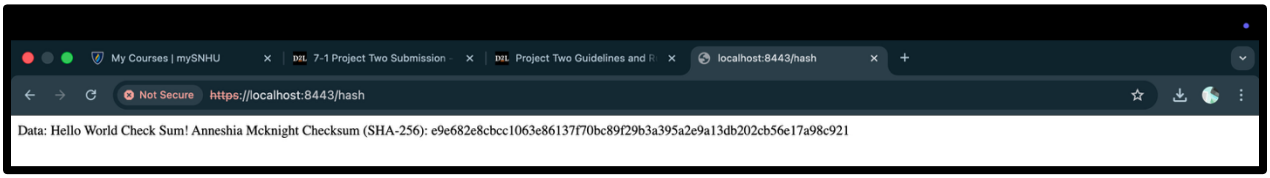
3. Deploy Cipher

Insert a screenshot below of the checksum verification.


My Courses | mySNHU
7-1 Project Two Submission
Project Two Guidelines and R
localhost:8443/hash
Not Secure
https://localhost:8443/hash
Data: Hello World Check Sum! Anneshia Mcknight Checksum (SHA-256): e9e682e8cbcc1063e86137f70bc89f29b3a395a2e9a13db202cb56e17a98c921

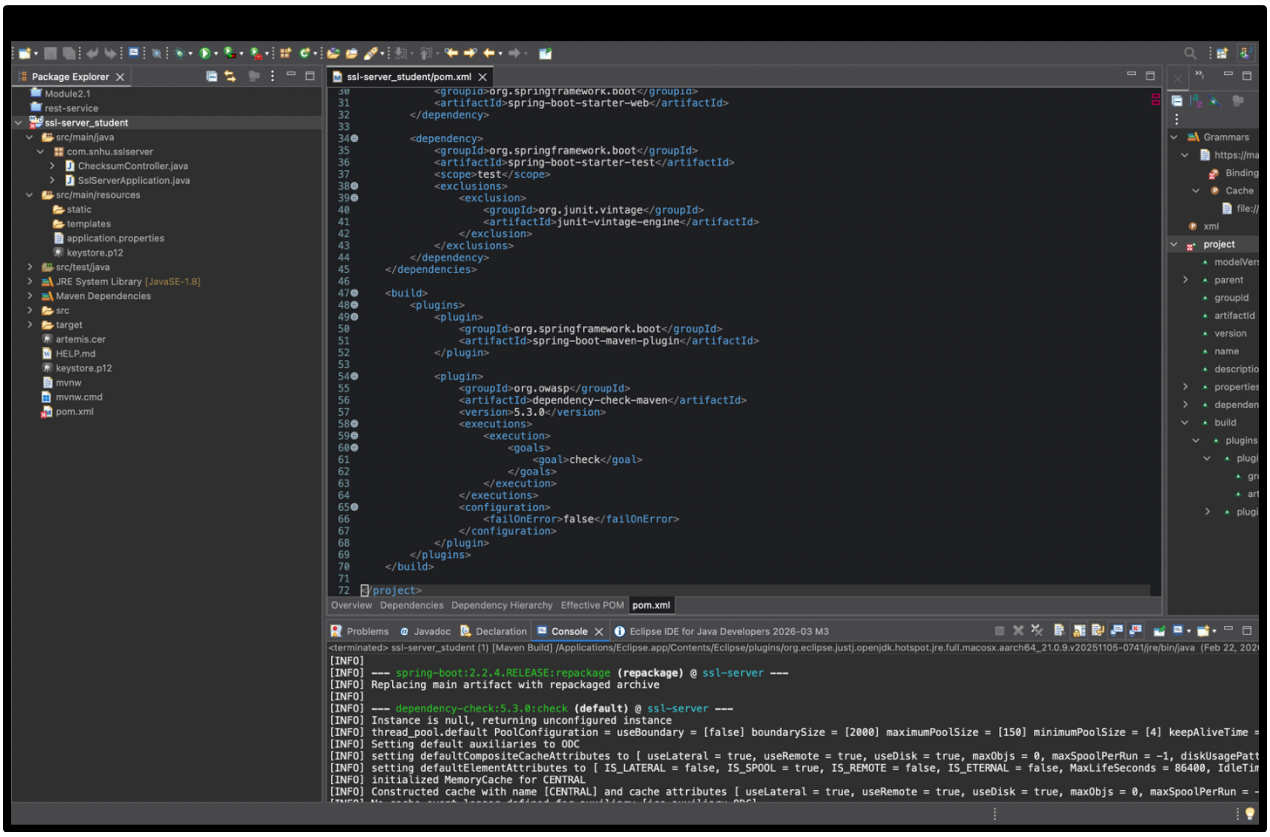
4. Secure Communications

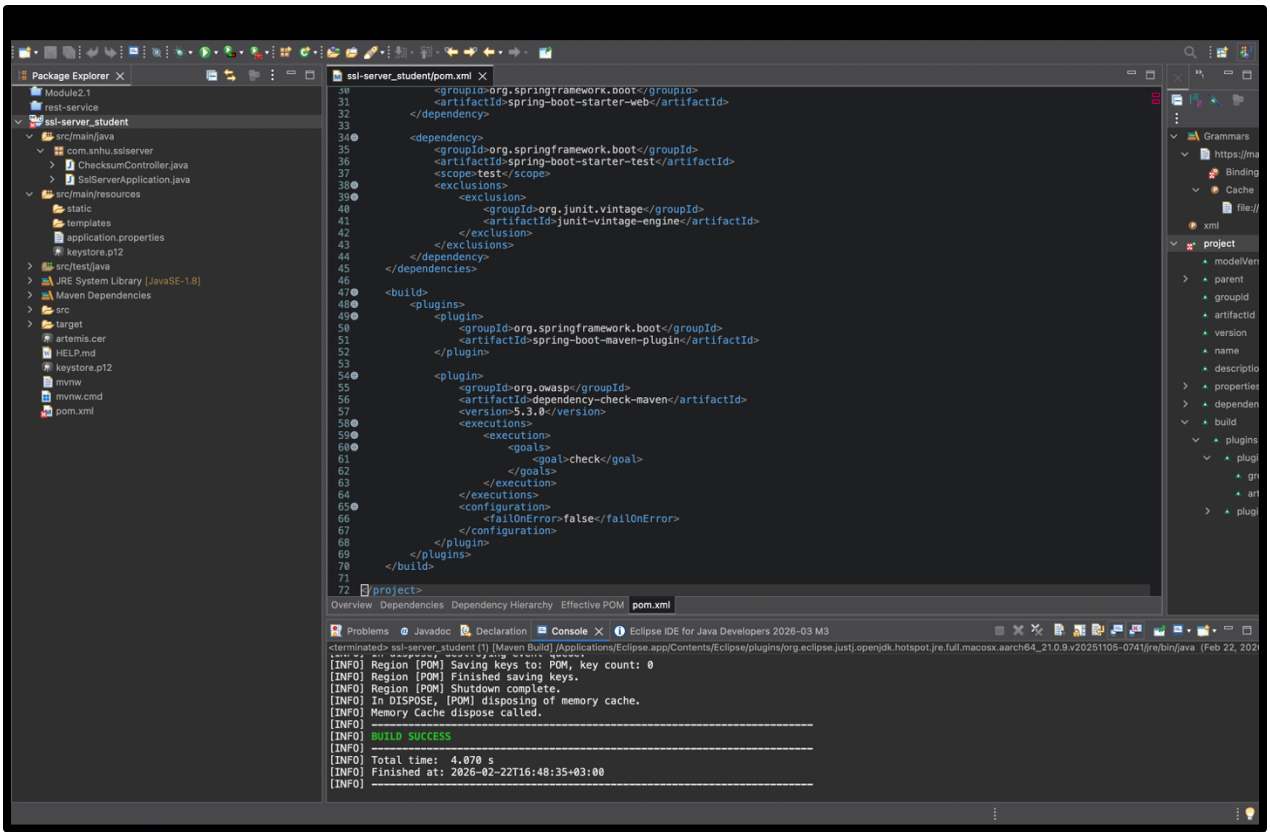
Insert a screenshot below of the web browser that shows a secure webpage.



5. Secondary Testing

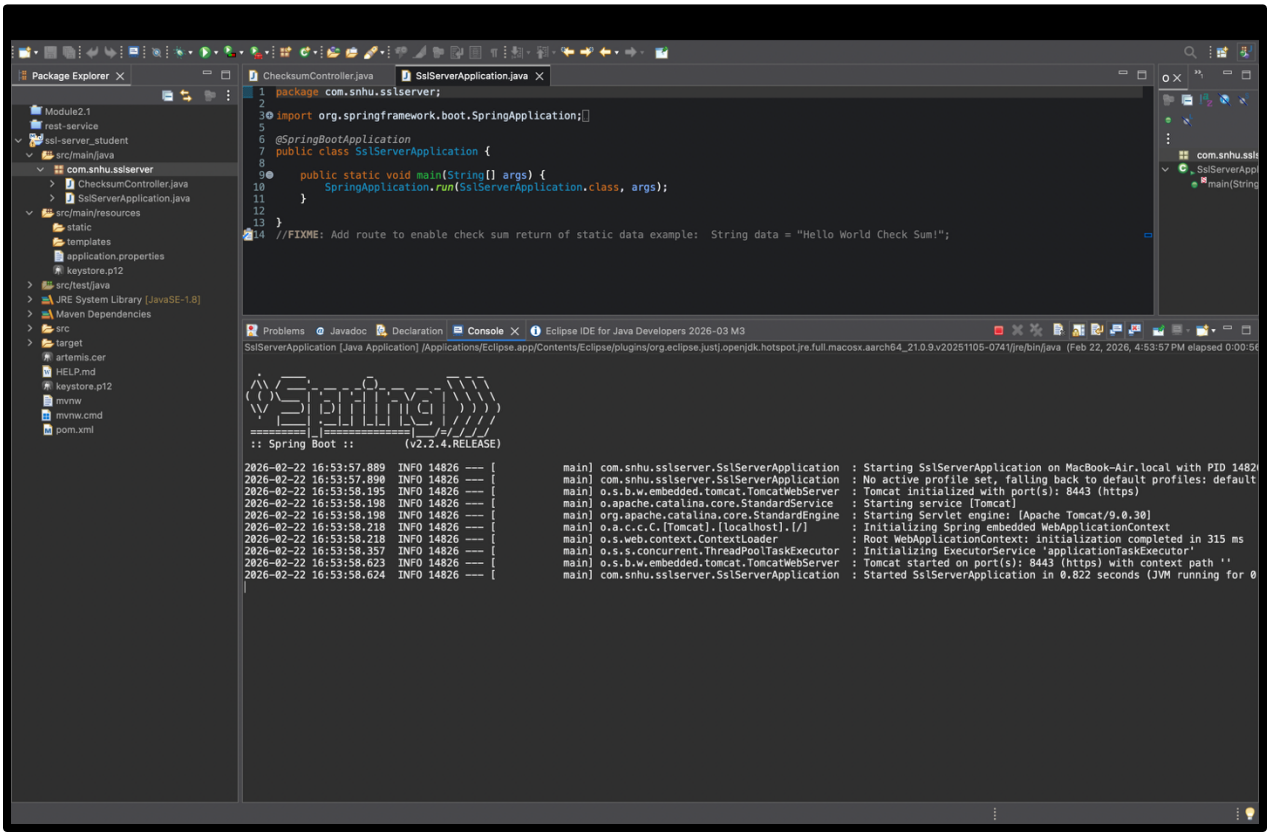
Insert screenshots below of the refactored code executed without errors and the dependency-check report.




Package Explorer
Module2.1
rest-service
ssl-server_student
src/main/java
com.snhu.sslserver
ChecksumController.java
SslServerApplication.java
src/main/resources
static
templates
application.properties
keystore.p12
src/test/java
JRE System Library [JavaSE-1.8]
Maven Dependencies
src
target
artemis.cer
HELP.md
keystore.p12
mvnw
mvnw.cmd
pom.xml
ssl-server_student/pom.xml
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
<exclusions>
<exclusion>
<groupId>org.junit.vintage</groupId>
<artifactId>junit-vintage-engine</artifactId>
</exclusion>
</exclusions>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
<plugin>
<groupId>org.owasp</groupId>
<artifactId>dependency-check-maven</artifactId>
<version>5.3.0</version>
<executions>
<execution>
<goals>
<goal>check</goal>
</goals>
</execution>
</executions>
<configuration>
<failOnError>false</failOnError>
</configuration>
</plugin>
</plugins>
</build>
</project>
Overview Dependencies Dependency Hierarchy Effective POM pom.xml
Problems Javadoc Declaration Console Eclipse IDE for Java Developers 2026-03 M3
<terminated> ssl-server_student (1) [Maven Build] /Applications/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.aarch64_21.0.9.v20251105-0741/jre/bin/java (Feb 22, 202
[INFO] Region [POM] Saving keys to: POM, key count: 0
[INFO] Region [POM] Finished saving keys.
[INFO] Region [POM] Shutdown complete.
[INFO] In DISPOSE, [POM] disposing of memory cache.
[INFO] Memory Cache dispose called.
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 4.070 s
[INFO] Finished at: 2026-02-22T16:48:35+03:00
[INFO] --

6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.



7. Summary

During the project, Artemis Financial was redesigned to meet secure communications requirements, a route for data hash verification using SHA-256 was added, a self-signed certificate was created, and application settings were updated. A separate dependency check was performed using OWASP Dependency-Check. Everything was tested, and the server runs over HTTPS, with no functional errors detected.

8. Industry Standard Best Practices

Secure coding practices were used during development, from the cryptographically strong SHA-256 algorithm to dependency checking using OWASP. Taken together, these measures help prevent data leaks and enhance trust in the system. Adherence to industry standards allows the company to maintain a robust architecture.